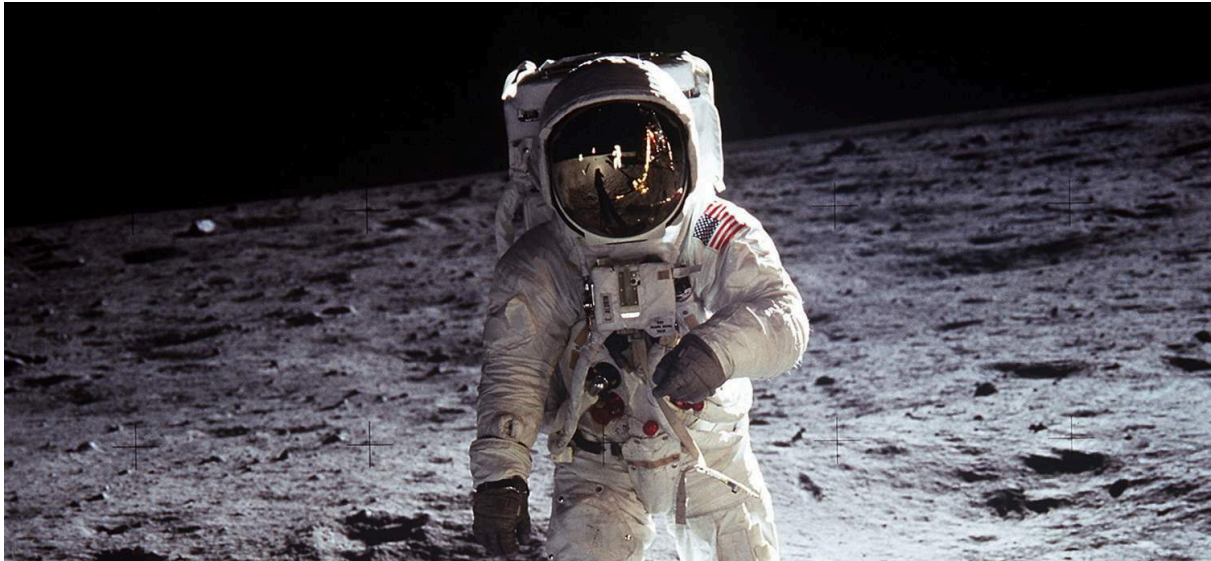




# Explanation of The Files



This project is developed for contact form mailing automation. GMAIL SMTP server is used mailing. I used docker image and container for the project.

## Purpose Of The Files In The Project

### 1. `.env`

The `.env` file (short for "environment") is used to store **configuration variables** specific to the environment the application is running in.

- **Purpose:** It keeps sensitive information and configuration settings separate from your main code and easy to change without modifying the application logic. This is crucial for following the **Twelve-Factor App** principles, specifically the principle of "Config in the environment."
- **Common Content:**
  - **Database credentials:** `DB_HOST` , `DB_USER` , `DB_PASSWORD`
  - **API keys:** `STRIPE_SECRET_KEY` , `GOOGLE_API_KEY`
  - **Application settings:** `PORT=3000` , `NODE_ENV=development`

- **Important Note:** The `.env` file should **always** be listed in your `.gitignore` file so it doesn't get accidentally committed to version control, protecting your secrets.
- 

## 2. `docker-compose.yml`

This file is used by **Docker Compose** to define and run multi-container Docker applications.

- **Purpose:** It allows you to define the services (containers), networks, and volumes for your entire application stack in a single file. This makes it easy to set up and run complex environments with a single command ( `docker-compose up` ).
  - **Common Content:**
    - **Service definitions:** Defining containers for your **application** (built from the `Dockerfile` ), a **database** (e.g., PostgreSQL, MongoDB), or a **caching service** (e.g., Redis).
    - **Dependencies:** Specifying which services depend on others.
    - **Volumes:** Defining how data persists (e.g., for the database).
    - **Environment variables:** Pulling values from the `.env` file for services.
- 

## 3. `Dockerfile`

The **Dockerfile** is a script containing a set of instructions that Docker uses to **build a Docker image**.

- **Purpose:** It defines exactly what your application's environment will look like, including the operating system, installed software, source code, and what command to run when the container starts. It creates a reproducible, portable environment for your application.
- **Common Content (for a Node.js app):**
  1. `FROM node:latest` : Specifies the base image (the starting point).
  2. `WORKDIR /usr/src/app` : Sets the working directory inside the container.
  3. `COPY package*.json ./` : Copies the dependency files.
  4. `RUN npm install` : Installs the dependencies.

5. `COPY ..` : Copies the rest of the application source code.
  6. `EXPOSE 3000` : Tells Docker the container listens on a certain port.
  7. `CMD [ "node", "server.js" ]` : The command to run the application when the container starts.
- 

## 4. `package.json`

This file is a standard part of all **Node.js** projects, created by the Node Package Manager (npm).

- **Purpose:** It holds **metadata** about your project and, most importantly, tracks all the **dependencies**(libraries/packages) your project needs to run.
  - **Common Content:**
    - `name` and `version` : Project identity.
    - `main` : The entry point file (often `server.js` ).
    - `scripts` : Command-line shortcuts for running tasks (like `start` , `test` , `dev` ).
    - `dependencies` : Libraries needed for the application to run in production (e.g., `express` ).
    - `devDependencies` : Libraries needed only for development and testing (e.g., `nodemon` , `jest` ).
- 

## 5. `server.js`

This is the main **application entry point** file, which contains your server's core logic.

- **Purpose:** It's the file executed by Node.js to start your web server. It sets up routes, middleware, and connects to other services like a database.
  - **Common Content (for an Express server):**
    - Importing necessary modules (like `express` ).
    - Initializing the Express application.
    - Defining application routes (e.g., `/users` , `/products` ).
    - Defining the port and starting the server to listen for incoming requests.
- 

In summary, these files work together like this:

1. `package.json` defines the code dependencies.
2. `server.js` is the actual application code.
3. `.env` holds the application's configuration settings.
4. The `Dockerfile` uses all of the above to build a portable application image.
5. The `docker-compose.yml` orchestrates running the application image alongside other services (like a database) in a complete development environment.



Java implementation can be found under  
EncryptorAndDecryptorImplementation folder



## References

### Google Gemini

Meet Gemini, Google's AI assistant. Get help with writing, planning, brainstorming, and more. Experience the power of generative AI.

 <https://gemini.google.com/>



## Author → Serhat Kumas

<https://www.linkedin.com/in/serhatkumas/>

### SerhatKumas - Overview

Computer engineering student who loves coding in different fields instead of focusing on a one spesific area. -

SerhatKumas

 <https://github.com/SerhatKumas>

